

Memorando

De: Emanuel Carneiro dos Santos

Nº de Matrícula: 20230429

Disciplina: Computação Paralela e Distribuída

Assunto: Introdução ao OpenMP

Data: 04/04/2026

1. Introdução

O presente laboratório tem como principal objectivo a introdução ao modelo de programação paralela utilizando OpenMP, com foco na criação, gestão e sincronização de threads em ambientes de memória partilhada.

O OpenMP é uma API para programação paralela em memória partilhada, compatível com C, C++ e Fortran, que assenta em três pilares: **directivas de compilação** (`#pragma omp`), **funções de biblioteca** (`<omp.h>`) e **variáveis de ambiente** como `OMP_NUM_THREADS`. O seu modelo de execução segue o paradigma *fork-join* — o programa inicia com uma única thread (master) que, ao encontrar uma região paralela, cria um grupo de threads (fork); no final dessa região, todas sincronizam e a execução retorna à thread master (join). A integração num programa C existente requer apenas a flag de compilação `-fopenmp` e a anotação das regiões paralelas com directivas como `#pragma omp parallel`, `#pragma omp for` e `#pragma omp barrier`. Por ser restrito à memória partilhada, o OpenMP opera numa única máquina, sendo complementado por MPI em ambientes distribuídos.

São dois os problemas abordados: o **Problema 1** explora a sincronização entre threads e o efeito de directivas como `nowait` e `barrier`; o **Problema 2** foca a paralelização de um loop interno com dependências de dados e as estratégias para garantir a correcção do resultado paralelo.

2. Experiências Realizadas

Problema 1

Este problema tenta familiarizar os estudante com o ambiente de trabalho e a compilação do código OpenMP. Considere o seguinte programa (omp1.c):

```
1. int main(int argc, char *argv[]) {
2.
3.     int i, tid;
4.
5.     #pragma omp parallel private(i,tid)
6.     {
7.         tid = omp_get_thread_num();
8.
9.     #pragma omp for
10.        for(i = 0; i < NUMITER; i++)
11.            printf("Thread: %d\titer=%d\n", tid, i); fflush(stdout);
12.
13.        printf("Thread %d, almost...\n", tid); fflush(stdout);
14.        printf("Thread %d, done!\n", tid); fflush(stdout);
15.    }
16.
17.    printf("All threads have finished!\n");
18. }
```

- a) Comece por compilar o código acima, não esqueça de adicionar a flag de compilação “fopenmp”. Execute este programa com um número diferente de threads (definindo OMP_NUM_THREADS).

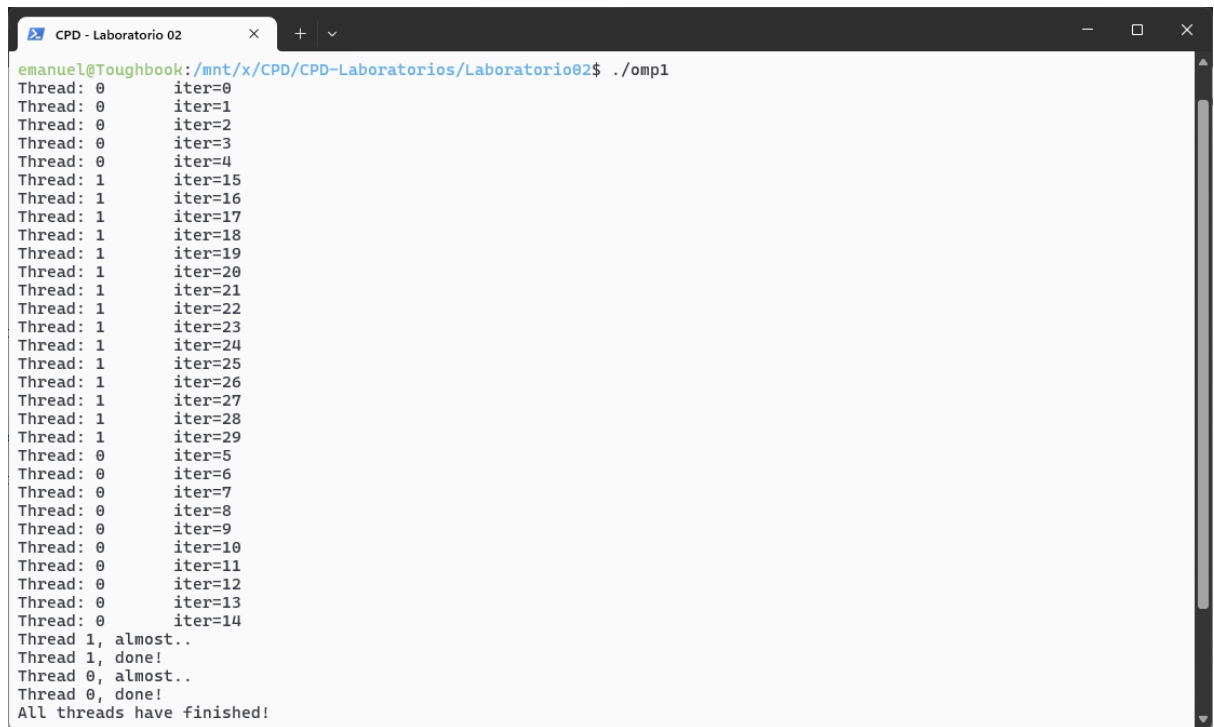
R:

O código foi compilado com o seguinte comando:

```
| gcc -fopenmp omp1.c -o omp1
```

O programa foi testado com diferentes valores de OMP_NUM_THREADS (2, 4 e 8):

```
| export OMP_NUM_THREADS=2
| ./omp1
```

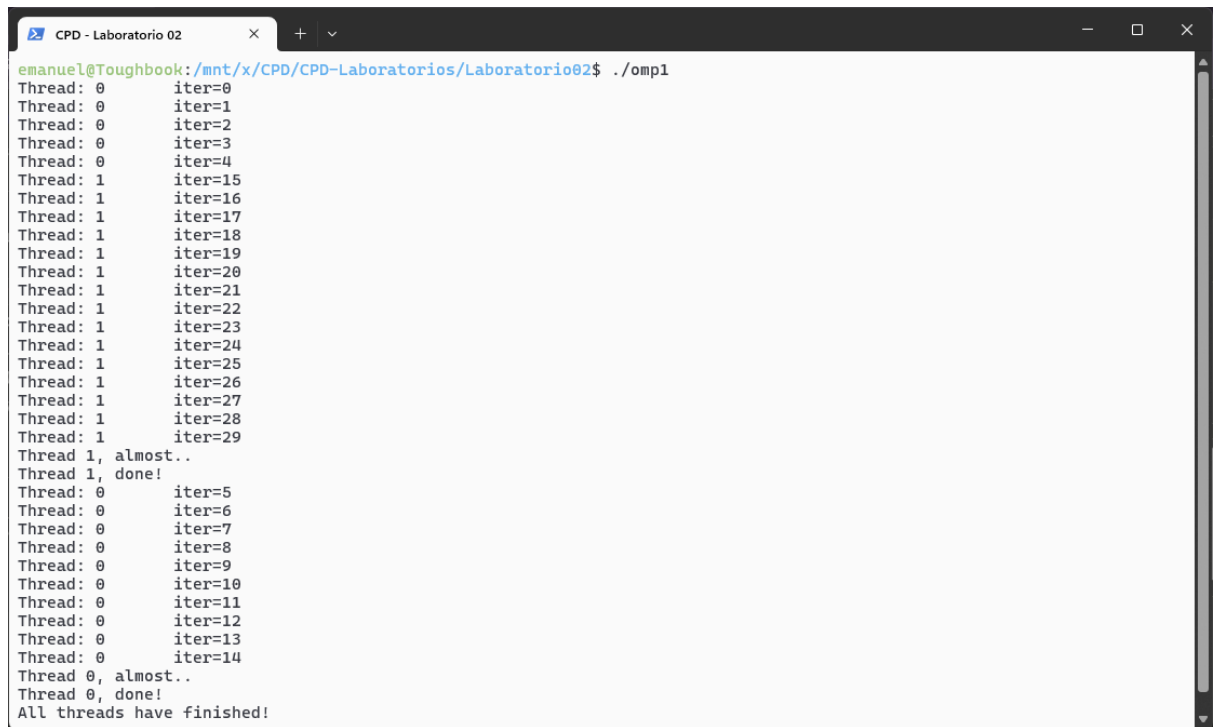


```
CPD - Laboratorio 02
emmanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio02$ ./omp1
Thread: 0      iter=0
Thread: 0      iter=1
Thread: 0      iter=2
Thread: 0      iter=3
Thread: 0      iter=4
Thread: 1      iter=15
Thread: 1      iter=16
Thread: 1      iter=17
Thread: 1      iter=18
Thread: 1      iter=19
Thread: 1      iter=20
Thread: 1      iter=21
Thread: 1      iter=22
Thread: 1      iter=23
Thread: 1      iter=24
Thread: 1      iter=25
Thread: 1      iter=26
Thread: 1      iter=27
Thread: 1      iter=28
Thread: 1      iter=29
Thread: 0      iter=5
Thread: 0      iter=6
Thread: 0      iter=7
Thread: 0      iter=8
Thread: 0      iter=9
Thread: 0      iter=10
Thread: 0      iter=11
Thread: 0      iter=12
Thread: 0      iter=13
Thread: 0      iter=14
Thread 1, almost..
Thread 1, done!
Thread 0, almost..
Thread 0, done!
All threads have finished!
```

Figura 1 - Execução do `omp1.c` com `OMP_NUM_THREADS=2`

Com 2 threads, o OpenMP dividiu as 30 iterações entre a Thread 0 (iteraões 0–14) e a Thread 1 (iteraões 15–29). A ordem de impressão reflecte o escalonamento do sistema: a Thread 1 começou a imprimir antes de a Thread 0 terminar o seu bloco, o que evidencia a execução verdadeiramente paralela e não-determinística. A barreira implícita no final do `#pragma omp for` garantiu, no entanto, que ambas as mensagens "almost.." e "done!" surgissem apenas após todas as iterações terem sido concluídas.

- b) Adicione a cláusula “`nowait`” à directiva `for` na linha 9. A saída foi alterada? Porquê?
R:



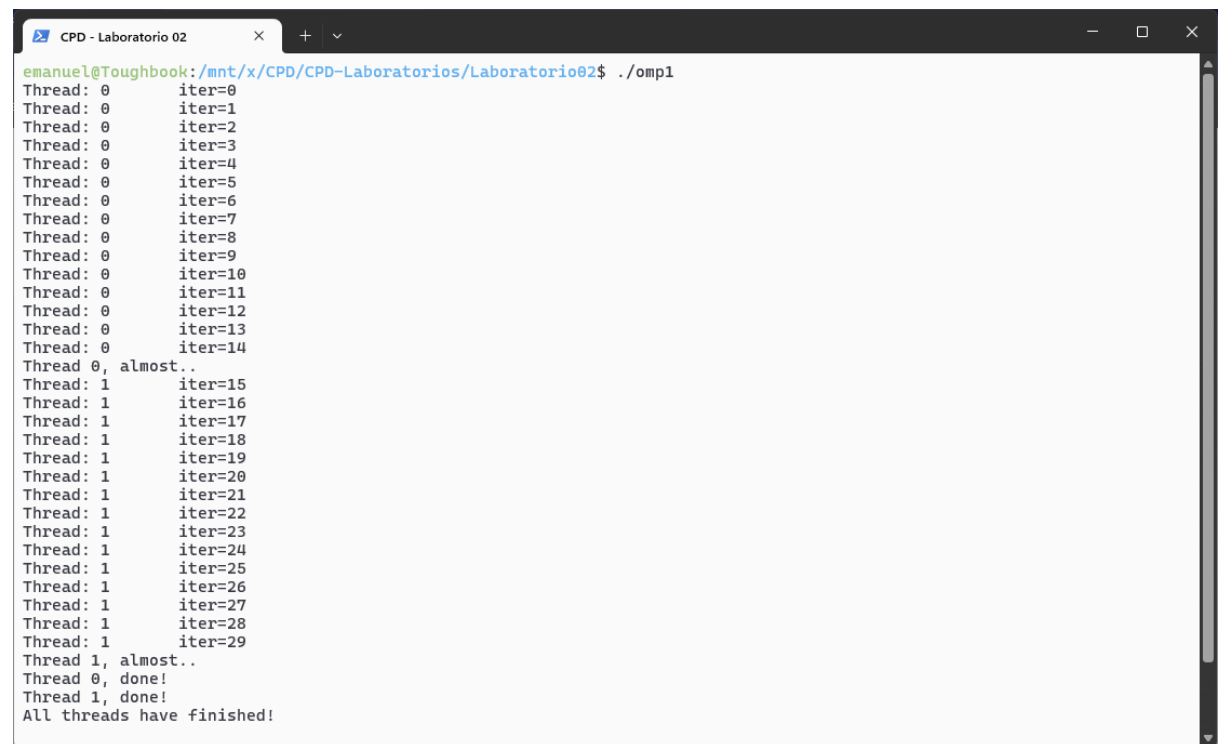
```
CPD - Laboratorio 02
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio02$ ./omp1
Thread: 0      iter=0
Thread: 0      iter=1
Thread: 0      iter=2
Thread: 0      iter=3
Thread: 0      iter=4
Thread: 1      iter=15
Thread: 1      iter=16
Thread: 1      iter=17
Thread: 1      iter=18
Thread: 1      iter=19
Thread: 1      iter=20
Thread: 1      iter=21
Thread: 1      iter=22
Thread: 1      iter=23
Thread: 1      iter=24
Thread: 1      iter=25
Thread: 1      iter=26
Thread: 1      iter=27
Thread: 1      iter=28
Thread: 1      iter=29
Thread 1, almost..
Thread 1, done!
Thread: 0      iter=5
Thread: 0      iter=6
Thread: 0      iter=7
Thread: 0      iter=8
Thread: 0      iter=9
Thread: 0      iter=10
Thread: 0      iter=11
Thread: 0      iter=12
Thread: 0      iter=13
Thread: 0      iter=14
Thread 0, almost..
Thread 0, done!
All threads have finished!
```

Figura 2 - Código com `nowait` e resultado da execução com `OMP_NUM_THREADS=2`

Com a cláusula `nowait`, a Thread 1 concluiu as suas iterações (15–29) e imprimiu imediatamente “almost..” e “done!” enquanto a Thread 0 ainda estava a executar as suas iterações (5–14). Isto demonstra com clareza o efeito do `nowait`: ao remover a barreira implícita do `#pragma omp for`, cada thread avança para as instruções seguintes assim que termina as suas próprias iterações, sem aguardar pelas restantes. Sem `nowait`, este comportamento seria impossível — a Thread 1 seria forçada a esperar pela Thread 0 antes de qualquer uma imprimir “almost..”.

- c) Adicione uma barreira (`#pragma omp barrier`) entre as linhas 13 e 14. Compare a saída com a) e b).

R:



```
CPD - Laboratorio 02
emanuel@Toughbook:/mnt/x/CPD/CPD-Laboratorios/Laboratorio02$ ./omp1
Thread: 0      iter=0
Thread: 0      iter=1
Thread: 0      iter=2
Thread: 0      iter=3
Thread: 0      iter=4
Thread: 0      iter=5
Thread: 0      iter=6
Thread: 0      iter=7
Thread: 0      iter=8
Thread: 0      iter=9
Thread: 0      iter=10
Thread: 0      iter=11
Thread: 0      iter=12
Thread: 0      iter=13
Thread: 0      iter=14
Thread 0, almost..
Thread: 1      iter=15
Thread: 1      iter=16
Thread: 1      iter=17
Thread: 1      iter=18
Thread: 1      iter=19
Thread: 1      iter=20
Thread: 1      iter=21
Thread: 1      iter=22
Thread: 1      iter=23
Thread: 1      iter=24
Thread: 1      iter=25
Thread: 1      iter=26
Thread: 1      iter=27
Thread: 1      iter=28
Thread: 1      iter=29
Thread 1, almost..
Thread 0, done!
Thread 1, done!
All threads have finished!
```

Figura 3 - Código com `#pragma omp barrier` e resultado da execução com `OMP_NUM_THREADS=2`

Problema 2

No segundo problema, o estudante é solicitado a paralelizar um loop interno com dependências de dados. O código básico que deve ser executado em paralelo é o seguinte:

```
for(iter = 0; iter < numiter; iter++)
{
    for (i=0; i<size-1; i++)
    {
        V[i] = f(V[i], V[i+1]);
    }
}
```

onde definimos uma função simples para manipular os dados.

```
#define f(x,y) ((x+y)/2.0)
```

a) Compile e execute a versão serial do código (omp2.c).

R:

O código foi compilado e executado sem paralelismo:

```
| cc omp2.c -o omp2 && ./omp2
```

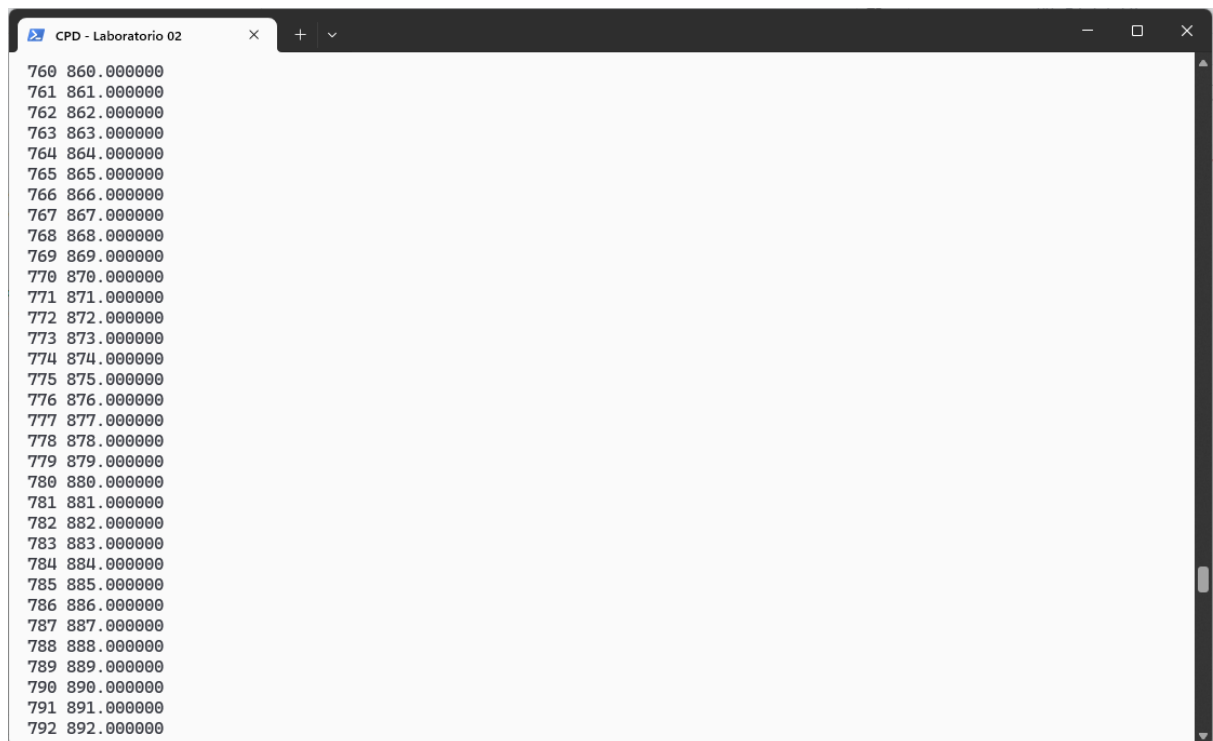


Figura 4 - Execução da versão serial do omp2.c (valores de referência)

```
real    0m0.037s
user    0m0.004s
sys     0m0.029s
```

O vector V é inicializado com valores de 0.0 a 999.0. Após 200 iterações do loop externo, os valores finais constituem a referência correcta para comparação com a versão paralela.

- b) Crie uma versão paralela simples (e incorreta) adicionando uma directiva paralela ao loop interno. Tente encontrar uma execução com uma saída diferente da versão serial. **Porquê isso pode acontecer?**

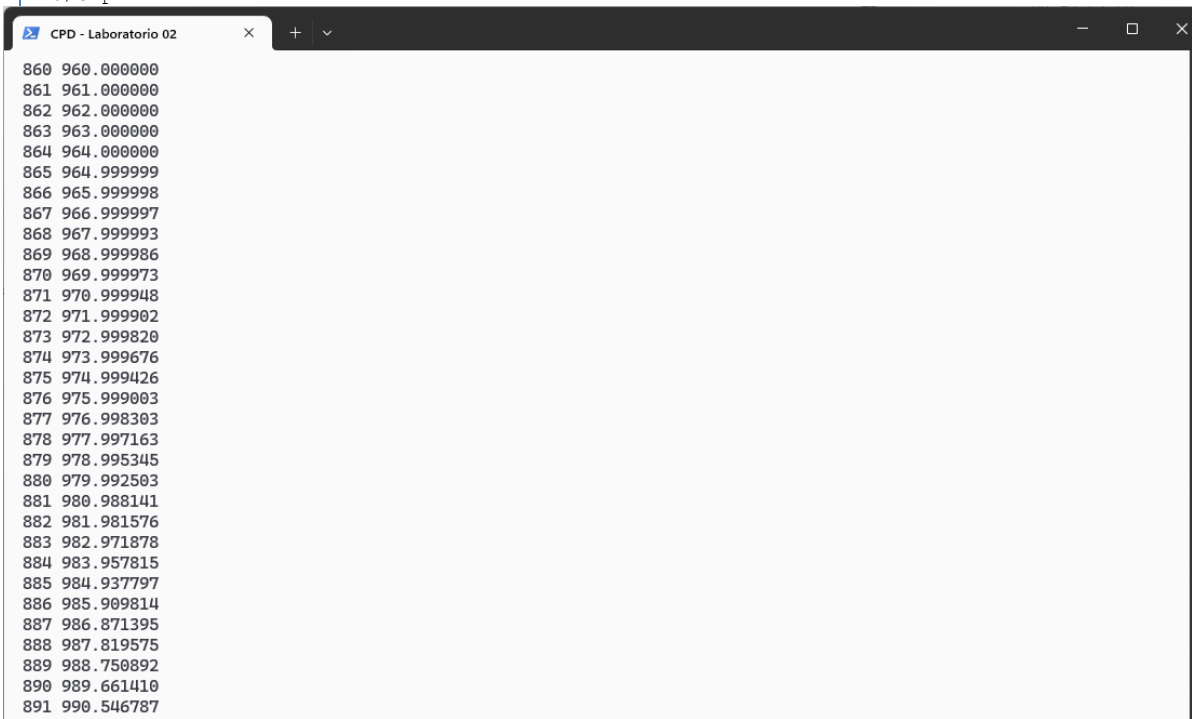
R:

Foi adicionada a directiva `#pragma omp parallel for` ao loop interno, sem qualquer mecanismo de sincronização adicional:

```
| #pragma omp parallel for
```

O código foi compilado e executado com paralelismo (incorretamente):

```
cc -fopenmp omp2.c -o omp2
export OMP_NUM_THREADS=2
./omp2
```



```
860 960.000000
861 961.000000
862 962.000000
863 963.000000
864 964.000000
865 964.999999
866 965.999998
867 966.999997
868 967.999993
869 968.999986
870 969.999973
871 970.999948
872 971.999902
873 972.999820
874 973.999676
875 974.999426
876 975.999003
877 976.998303
878 977.997163
879 978.995345
880 979.992503
881 980.988141
882 981.981576
883 982.971878
884 983.957815
885 984.937797
886 985.909814
887 986.871395
888 987.819575
889 988.750892
890 989.661410
891 990.546787
892 991.402376
```

Figura 5 – Execução da versão paralela incorrecta do `omp2.c` e divergência face à versão serial

Esta versão produz resultados incorrectos porque existe uma condição de corrida (race condition): a thread que processa a iteração i pode ler `v[i+1]` antes de a thread responsável pela iteração $i+1$ ter actualizado esse valor — ou vice-versa. A ordem de execução das threads não é garantida, pelo que o resultado depende do escalonamento e é não-determinístico.

- c) Faça a correcção da versão paralela copiando o vector V da iteração $i-1$ antes de executar a iteração i e usando os valores copiados como argumentos para a função f na iteração i .

R:

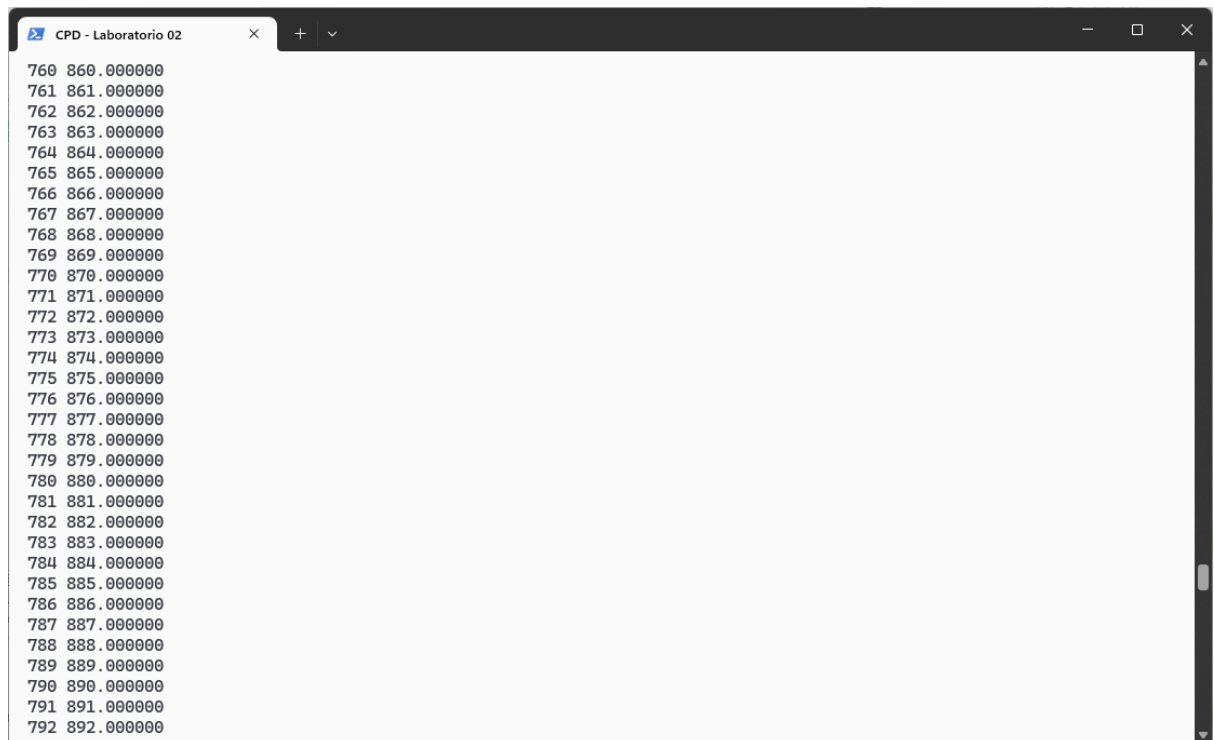
A correcção consiste em copiar o vector v no início de cada iteração do loop externo e usar os valores copiados (V_{copy}) como argumentos da função f , enquanto os resultados são escritos em v . Desta forma, a leitura de $v[i+1]$ é sempre feita a partir da cópia, que permanece inalterada durante toda a iteração paralela do loop interno.

```
35  /* DECLARE VECTOR AND AUX DATA STRUCTURES */
36  double *Vcopy = (double *) malloc(TOTALSIZE * sizeof(double));
37  double *V = (double *) malloc(TOTALSIZE * sizeof(double));
38
39  /* 2. ITERATIONS LOOP */
40  for(iter = 0; iter < NUMITER; iter++) {
41      memcpy(Vcopy, V, TOTALSIZE * sizeof(double));
42
43      /* 2.1. PROCESS ELEMENTS */
44      #pragma omp parallel for
45      for(i = 0; i < TOTALSIZE-1; i++) {
46          V[i] = f(Vcopy[i], Vcopy[i+1]);
47      }
48
49      /* 2.2. END ITERATIONS LOOP */
50  }
```

Figura 6 - Versão paralela correcta com cópia do vector

O código foi compilado e executado com os seguintes comandos:

```
cc omp2.c -o omp2
export OMP_NUM_THREADS=2
./omp2
```



```
760 860.000000
761 861.000000
762 862.000000
763 863.000000
764 864.000000
765 865.000000
766 866.000000
767 867.000000
768 868.000000
769 869.000000
770 870.000000
771 871.000000
772 872.000000
773 873.000000
774 874.000000
775 875.000000
776 876.000000
777 877.000000
778 878.000000
779 879.000000
780 880.000000
781 881.000000
782 882.000000
783 883.000000
784 884.000000
785 885.000000
786 886.000000
787 887.000000
788 888.000000
789 889.000000
790 890.000000
791 891.000000
792 892.000000
```

Figura 7 – Execução Versão paralela correcta e confirmação da igualdade de resultados

Nesta versão, o resultado da execução paralela é idêntico ao da versão serial. A cópia garante que todas as threads lêem os valores de *V* do início da iteração, eliminando a condição de corrida.

```
real    0m0.031s
user    0m0.008s
sys     0m0.022s
```

- d) Escreva uma modificação na versão paralela que evite copiar o vetor *V* em todas as iterações.

R:

Para evitar a cópia do vetor em todas as iterações, foi utilizada uma abordagem baseada em **duplo buffer**, recorrendo a dois vetores distintos para **leitura** e **escrita**. Em cada iteração, os cálculos são efectuados a partir do vetor fonte (*src*) e armazenados no vetor destino (*dst*), eliminando dependências de dados entre threads. Após o processamento, os ponteiros dos vetores são trocados, evitando operações de cópia e reduzindo o custo computacional. Adicionalmente, o último elemento do vetor é tratado separadamente para garantir a consistência dos dados.

```
34  /* DECLARE VECTOR AND AUX DATA STRUCTURES */
35  double *src = (double *) malloc(TOTALSIZE * sizeof(double));
36  double *dst = (double *) malloc(TOTALSIZE * sizeof(double));
37  double *tmp;
```

```

44  /* 2. ITERATIONS LOOP */
45  for(iter = 0; iter < NUMITER; iter++) {
46      /* 2.1. PROCESS ELEMENTS */
47      #pragma omp parallel for
48      for(i = 0; i < TOTALSIZE-1; i++) {
49          dst[i] = f(src[i], src[i+1]);
50      }
51      dst[TOTALSIZE-1] = src[TOTALSIZE-1];
52      tmp = src;
53      src = dst;
54      dst = tmp;
55      /* 2.2. END ITERATIONS LOOP */
56  }

```

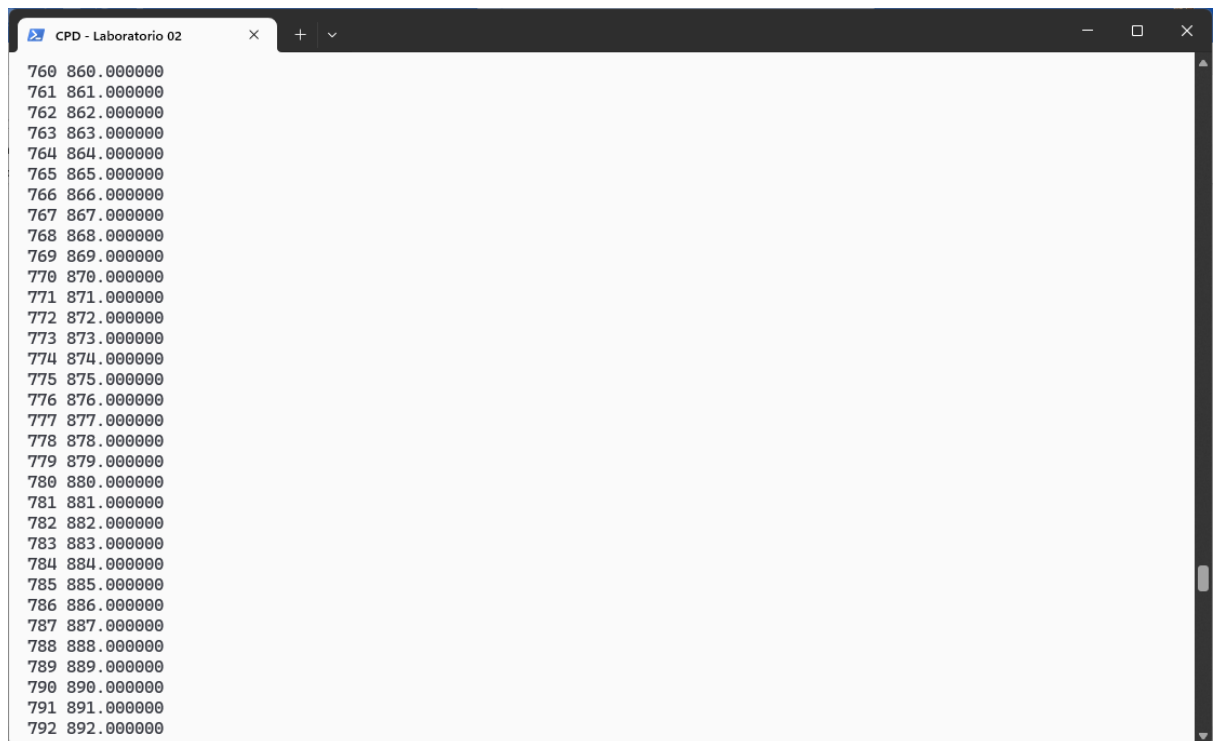
Figura 8 - Versão paralela otimizada com double buffering (sem cópia explícita por iteração)

O código foi compilado e executado com os seguintes comandos:

```

cc omp2.c -o omp2
export OMP_NUM_THREADS=2
./omp2

```



```

760 860.000000
761 861.000000
762 862.000000
763 863.000000
764 864.000000
765 865.000000
766 866.000000
767 867.000000
768 868.000000
769 869.000000
770 870.000000
771 871.000000
772 872.000000
773 873.000000
774 874.000000
775 875.000000
776 876.000000
777 877.000000
778 878.000000
779 879.000000
780 880.000000
781 881.000000
782 882.000000
783 883.000000
784 884.000000
785 885.000000
786 886.000000
787 887.000000
788 888.000000
789 889.000000
790 890.000000
791 891.000000
792 892.000000

```

Figura 9 – Execução da versão paralela otimizada com double buffering

```

real    0m0.021s
user    0m0.014s
sys     0m0.006s

```

Esta solução é mais eficiente do que a versão c) porque elimina a operação de `memcpy` a cada iteração ($O(N)$ por iter), substituindo-a por uma simples troca de ponteiros ($O(1)$). O resultado final permanece correcto e idêntico à versão serial.

3. Desafios

Ao longo da realização desta aula de laboratório, foram identificados os seguintes desafios:

Compilar o programa: ao compilar um programa que possui funções da biblioteca `<omp.h>` sem a flag de compilação `-fopenmp` geralmente dá o seguinte erro:

```
/usr/bin/ld: /tmp/ccpwMbXP.o: in function `main':  
ompl.c:(.text+0x14): undefined reference to `omp_get_thread_num'  
collect2: error: ld returned 1 exit status
```

Identificar a condição de corrida no Problema 2: foi necessário executar o programa várias vezes e comparar os resultados com a versão serial para confirmar a incorrecção da versão paralela simples.

Apesar da paralelização implementada no problema 2, não se observou um ganho significativo de desempenho para um número elevado de threads. Tal comportamento deve-se ao reduzido volume de trabalho por iteração e ao overhead associado à gestão de threads no OpenMP.

4. Referências Bibliográficas

OpenMP Architecture Review Board. *OpenMP Application Programming Interface, Version 4.5*. Novembro 2015. Disponível em: <https://www.openmp.org/wp-content/uploads/openmp-4.5.pdf>

Costa, J. *Computação Paralela e Distribuída – Aula de Laboratório Nº 2*. ISPTEC, 2021-22.

Chapman, B., Jost, G., van der Pas, R. *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press, 2008.

5. Repositório GitHub

Repositório: github.com/SW-Wanted/CPD-Laboratorios.¹

¹ Foi enviado um convite de colaborador ao utilizador [joaojdacosta](#).